

WILLMA APK Analysis Report

Barcode, Label & Food Scanning Feature Workflow

Reverse-Engineered from WILLMA v1.10.1 (com.willma.client)
XAPK Package from APKPure

Package Name	com.willma.client
App Name	WILLMA
Version	1.10.1 (Build 21)
Min SDK	23 (Android 6.0)
Target SDK	35 (Android 15)
Framework	Expo React Native (SDK 51)
API Endpoint	https://app.willma.life/api/graphql
AI Provider	Anthropic Claude (claude-opus-4-20250514-v1:0)
Barcode Engine	Google MLKit Barcode Scanner (TFLite)

1. Executive Summary

WILLMA is a health and nutrition mobile application built with Expo React Native (SDK 51) that provides AI-powered food scanning, barcode recognition, and nutrition tracking capabilities. The app integrates Google MLKit for on-device barcode detection and Anthropic Claude AI (specifically the claude-opus-4-20250514-v1:0 model) for intelligent meal analysis. The scanning feature is gated behind a premium subscription model, where free users have limited daily scans and premium users enjoy unlimited scanning. The app is particularly tailored for Middle Eastern and Ramadan-specific dietary guidance, with built-in awareness of Arabic food names, cooking methods, and religious fasting schedules.

The app's scanning system operates through two primary pathways: a barcode scanning path that uses MLKit's on-device TFLite models to detect and decode product barcodes (supporting GTIN, UPC, EAN formats) and then looks up the product in a connected food database; and a photo-based AI analysis path where the user captures an image of their meal or a nutrition label, which is then uploaded to the backend and processed by Anthropic Claude to identify individual ingredients, estimate portion sizes, and calculate comprehensive nutritional profiles including calories, protein, carbohydrates, fat, fiber, sodium, sugar, and a health score rating.

Key technical findings include: the app uses GraphQL (Apollo Client) for all backend communication, stores nutrition data locally in SQLite with tables for meal tracking, ingredient management, and nutrition plan management, and implements a sophisticated streak/gamification system to encourage daily meal logging. The AI analysis is contextually aware, receiving not just the food image but also the user's profile, previously eaten meals that day, remaining daily calorie budget, and Ramadan phase information to provide personalized dietary recommendations.

2. Application Architecture

2.1 Technology Stack

Layer	Technology	Details
Frontend Framework	Expo React Native SDK 51	TypeScript, Expo Router (file-based routing)
Styling	Tailwind CSS + Custom	NativeWind/Tailwind with AGBQ font family
State Management	Apollo Client + Redux	GraphQL cache + local state, MMKV storage
API Layer	GraphQL (Apollo)	https://app.willma.life/api/graphql
Camera	Expo CameraView	Photo capture, torch, mirror mode
Barcode Scanner	Google MLKit	TFLite models, on-device inference
AI Engine	Anthropic Claude Opus 4	claude-opus-4-20250514-v1:0
Local Database	SQLite (WatermelonDB)	8 DEX files, 5.8M-9.7M each
Analytics	Sentry + Facebook SDK	Error tracking, user analytics
Payments	RevenueCat / IAP	Premium subscription gating

2.2 Permissions

The app requests the following key permissions: CAMERA (for food photo capture and barcode scanning), INTERNET (for GraphQL API calls and AI analysis), ACCESS_FINE_LOCATION and ACCESS_COARSE_LOCATION (for Ramadan prayer time calculations and nearby mosque information), READ_EXTERNAL_STORAGE and WRITE_EXTERNAL_STORAGE (for image handling), RECORD_AUDIO (for chat voice messages), VIBRATE and POST_NOTIFICATIONS (for streak reminders and engagement notifications), and BILLING (for premium subscription purchases). The camera permission is the most critical for the scanning features, as it enables both the live barcode detection feed and the photo capture for AI meal analysis.

3. Barcode Scanning Workflow

3.1 Entry Point and Premium Gating

The barcode scanning feature is accessed from the home screen via prominent call-to-action buttons displaying motivational text such as "Scan your food" or "Scan your lunch to stay sharp and on track." Before the camera is opened, the app performs two critical checks: first, it queries the GraphQL backend with the HasPremiumFeature query, passing the feature parameter "meal_scan" to determine if the user has an active premium subscription. If the user is not a premium subscriber, a paywall is displayed with messages like "To continue meal scanning, subscribe to premium" or "Upgrade for Unlimited Scans." Second, it checks the local flag HAS_SEEN_MEAL_SCAN_INTRO to determine if the user has previously viewed the onboarding carousel that explains how the scanning feature works. First-time users see an animated introduction with progress indicators before being granted access to the camera.

The app also tracks a daily scan count that increments each time a meal scan is performed. Free users have a limited number of daily scans, while premium users enjoy unlimited scanning. After a certain number of successful scans, the app may prompt the user to leave an App Store review, using the `_maybeRequestAppReview` mechanism. The scan count is stored on the user profile and persisted across sessions.

3.2 Camera and Barcode Detection

Once the premium and onboarding checks pass, the app opens the device camera using the Expo CameraView component. The camera interface includes torch/flashlight support, a mirror mode toggle, and a photo capture button. The app advises users to "take your meal photo at a tilted angle for more accuracy when scanning the amounts," suggesting that the AI analysis performs better with angled photos that capture food depth and portion sizes more accurately.

The barcode detection runs in real-time using Google MLKit's on-device barcode scanner. The app includes three TFLite models bundled in the `assets/mlkit_barcode_models/` directory: `barcode_ssd_mobilenet_v1_dmp25_quant.tflite` (390KB, for barcode detection using SSD MobileNet), `oned_auto_regressor_mobile.tflite` (214KB, for 1D barcode boundary regression), and `oned_feature_extractor_mobile.tflite` (277KB, for 1D barcode feature extraction). These models run entirely on-device, enabling fast barcode detection without requiring network connectivity. The barcode scanner supports multiple formats including Code128 and standard GTIN/UPC/EAN codes. Two configuration flags control the barcode scanner: `barcodeScannerEnabled` (master toggle) and `barcodeScannerSetting` (configuration options), with `isModernBarcodeScannerAvailable` checking device

compatibility before activation.

3.3 Product Lookup (Barcode Path)

When a barcode is successfully detected and decoded, the app extracts the GTIN/UPC/EAN code and sends it to the backend via the ScanProduct GraphQL mutation. This mutation accepts a ScanProductInput parameter containing the barcode data and returns the product name. The backend looks up the product in a connected food and product database, which includes an online store/supermarket integration with routes for product pages (online-store/product-page), category browsing (online-store/category-products), brand listings (online-store/all-brands), and supermarket seller information (online-store/supermarket/product-seller). The product data structure includes fields for title, description, price, availability (in_stock, out_of_stock, preorder, etc.), condition (new, refurbished, used), brand, and GTIN identifier.

4. Food Photo and Nutrition Label Scanning Workflow

4.1 Image Capture

When no barcode is detected, or when the user deliberately chooses to photograph their meal, the app captures a photo using the takePictureAsync method from Expo Camera. Alternatively, users can select an existing image from their device gallery via the handlePickImage function, which uses the Expo image picker with optional cropping support. The captured or selected image is prepared for upload to the backend as part of the ScanMeal GraphQL mutation. The image is also stored locally and associated with the scanned meal record via the imageUrl field in the nutrition_day_meals SQLite table.

4.2 AI-Powered Meal Analysis

The core of the food scanning feature is the AI-powered meal analysis performed by Anthropic Claude. The app sends the ScanMeal mutation to the GraphQL backend with two parameters: scanMealInput (a ScanMealInput object containing metadata about the scan) and file (the uploaded image as a GraphQL Upload type). The backend processes the image using the claude-opus-4-20250514-v1:0 model with a carefully constructed prompt that includes the following contextual information to improve analysis accuracy:

Context Section	Description
MEAL NUTRITION FACTS	Current nutrition data extracted from the image (calories, carbs, protein, fat, fiber, sodium, sugar)
MEALS EATEN TODAY	List of previously logged meals for the day to avoid double-counting and provide cumulative tracking
PREVIOUS GUIDANCE	Prior AI recommendations given to the user, ensuring continuity and avoiding repetitive advice
REMAINING DAILY BUDGET	Calculated remaining calories and macros for the day based on the user's nutrition plan targets
USER PROFILE	User's goals, activity level, dietary preferences, allergies, and medical conditions

RAMADAN PHASE & TIMING	Current Ramadan status (fasting/Iftar/Suhoor), prayer times, and location-aware scheduling
INGREDIENTS	Extracted list of individual ingredients with portion estimates and per-ingredient nutrition data

Table 1: AI Context Sections for Meal Analysis

The AI returns a structured response containing the following fields: name (meal name), calories, carbohydrates, fat, fiber, healthScore (a numerical rating of the meal's nutritional quality), id (unique identifier), imageUrl, isScannedMeal (boolean flag set to true for AI-scanned meals), protein, sodium, sugar, time (meal time), and a scannedFoodIngredients array. Each ingredient in the array contains: calories, carbohydrates, fat, fiber, id, name, protein, quantity, sodium, sugar, unit, and isScannedProduct (a boolean that distinguishes between label-scanned products where isScannedProduct is true and AI-estimated meal ingredients where it is false).

4.3 Nutrition Label Processing

When the user scans a nutrition facts label from food packaging, the same ScanMeal mutation is used, but the AI is specifically instructed to extract structured data from the label. The AI parses the nutrition label to identify per-serving calorie counts, macro breakdowns, ingredient lists, and serving size information. A key distinction in the data model is the isScannedProduct field within scannedFoodIngredients, which is set to true when the ingredient data comes from a scanned nutrition label rather than from AI estimation of a photographed meal. This allows the app to present label-scanned data with higher confidence indicators and different editing options compared to AI-estimated data.

The AI is also trained to recognize region-specific food terminology. The app includes specific recognition patterns for Arabic cooking methods (grilled/fried/boiled/baked/roasted/omelette), bread types (baladi, fino, toast, shamsi, roti, tortilla), protein types (chicken, beef, mixed, lamb, fish, shrimp, eggs), and dairy/fat categories (gebna=cheese, laban=milk, zabadi=yogurt, eshta=cream, samna=ghee, zebda=butter, tahina=tahini, zayt=oil). This localization enables accurate analysis of Middle Eastern cuisine that general-purpose food recognition systems often struggle with.

4.4 Error Handling and Fallbacks

The app implements multiple layers of error handling for the scanning feature. If the AI response cannot be parsed into the expected data structure, the error "Failed to parse scanned data" is logged via Sentry. Similarly, "Failed to parse AI response" handles cases where the AI returns malformed content, and "Failed to parse additionalInfo JSON" covers metadata parsing failures. The error "Failed to scan meal" is a general catch-all for scan failures. Camera-specific errors include "Failed to open camera" and "Image picker not available." In all error cases, the app falls back to manual meal entry, where the user can input nutrition data directly by searching for foods, ingredients, or meals from the existing database.

5. Post-Scan Review and Logging Workflow

5.1 Results Display

After the scan completes, the app displays the results in a meal card component (at-meal-card) showing the meal name, total calorie count, macro breakdown (protein, carbohydrates, fat), a health score, and an

ingredient list. Each identified ingredient is rendered as an individual "Ingredient Card" with its own nutrition details. The barcode-scan-card-1 chart visualization provides a graphical representation of the nutrition data. During the processing phase, the app shows status messages like "Identifying individual ingredients..." and "Refining details with AI..." to keep the user informed. After logging, the message "Meal logged! Analyzing in background..." indicates that further AI analysis is proceeding asynchronously.

5.2 Editing and Correction

The app provides extensive editing capabilities after a scan. Users can adjust portion sizes with options like "Eat 3/4 Portion" and receive confirmation messages like "Portion adjusted successfully." The `adjust_portion` type allows for fine-grained portion control. Users can also add or remove individual ingredients (`AddIngredient`, `RemoveIngredient`), directly edit macro values ("Edit Macros"), modify the meal name (which must be at least 2 characters), and correct any misidentified items. When the user makes corrections, the `UpdateScannedMeal` mutation is called with an `UpdateScannedMealDto` input, which updates the scanned meal data on the backend. The "Applying your correction" feedback loop ensures that user corrections improve the system's accuracy over time.

5.3 Meal Logging and Nutrition Plan Integration

Once the user confirms the scan results, the meal is logged using the `LogMeal` mutation, which accepts a `CreateNutritionDayMealInput` and the current date. The meal is stored both on the backend and locally in the SQLite database. The local `nutrition_day_meals` table includes fields for: `id`, `nutritionPlanDayId`, `trainerId`, `name`, `sequence`, `calories`, `protein`, `carbohydrates`, `fat`, `isCheatMeal`, `isAddedByClient`, `status` (`Planned`/`Skipped`/`Logged`), `time`, `notes`, and the critical `isScannedMeal` flag that marks the meal as originating from a scan rather than manual entry. The scanned `FoodIngredients` are stored in the `meal_ingredients` table with foreign key references to the parent meal.

The logged meal is automatically integrated with the user's active nutrition plan. The `nutrition_plan_days` table tracks target vs actual calories and macros for each day, and the app calculates the remaining daily budget after each logged meal. The health score provides feedback on meal quality, and the streak system updates to reflect the user's consistency. The app tracks `logsToday` and `logsYesterday` on the user's streak object, with motivational messages like "2 meals today to thaw your streak and continue your run" and "Log today's meal and begin a new run - one step at a time."

6. Key Data Model

6.1 GraphQL Mutations

Mutation	Input Type	Purpose
ScanMeal	ScanMealInput + Upload	AI analysis of food photo/label; returns full nutrition data with ingredients
ScanProduct	ScanProductInput	Barcode-based product lookup from food database
UpdateScannedMeal	UpdateScannedMealDto	User corrections to scanned meal data
LogMeal	CreateNutritionDayMealInput	Log scanned meal to nutrition plan day

createPrivateMeal	CreateNutritionTemplateDay MealInput + Upload	Save scanned meal as reusable private meal
-------------------	--	--

Table 2: Key GraphQL Mutations for Scanning

6.2 Local SQLite Schema

The app uses a SQLite database managed by WatermelonDB with the following key tables relevant to scanning: The `nutrition_day_meals` table stores all logged meals with fields for `id`, `nutritionPlanDayId`, `trainerId`, `name`, `sequence`, `calories`, `protein`, `carbohydrates`, `fat`, `isCheatMeal`, `isAddedByClient`, `status` (default 'Planned'), `time`, `notes`, and `timestamps`. The `isScannedMeal` boolean flag distinguishes scanned meals from manually entered ones. The `meal_ingredients` table links ingredients to meals with fields for `id`, `mealId`, `sourceId`, `replacedWith`, `ingredientType`, `plannedQuantity`, `actualQuantity`, and `isAddedByClient`. The `foods` table stores the food reference database with `id`, `name`, `calories`, `fat`, `protein`, `carbohydrates`, `size`, `sizeUnitAbbreviation`, `image`, and `description`. The `nutrition_plan_days` table tracks daily nutrition targets and actuals with `targetDayCalories`, `targetDayProtein`, `targetDayCarb`, `targetDayFat`, and corresponding actual values.

7. Visual Workflow Diagram

The following diagram illustrates the complete barcode, label, and food scanning workflow, from user entry through AI analysis to meal logging and post-scan features. The workflow is organized into six phases: User Entry and Premium Gating, Camera Capture and Barcode Detection, Data Processing and AI Analysis, Scan Results Display and Editing, Meal Logging and Tracking, and Post-Logging Features and Intelligence.



Figure 1: WILLMA Barcode, Label, and Food Scanning Complete Workflow

8. Security and Configuration Findings

During the analysis, several security-relevant findings were discovered in the app's configuration. The app.config file embedded in the APK contains hardcoded API keys for multiple AI services, including a DeepSeek API key (sk-f4e0...), an OpenAI API key (sk-proj-...), a Google Gemini API key (AlzaSy...), and an Anthropic API key (sk-ant-api03-...). While the AI_PROVIDER is set to "anthropic" as the active provider, the presence of these keys in the client-side configuration is a security concern, as they could potentially be extracted and misused. The active AI provider is Anthropic, which is the only one actually used for meal analysis.

The app also includes a Sentry DSN for error tracking (pointing to sentry.io project willma-client-prod under the willma organization), a Facebook SDK integration with app ID 1241712067901568 for analytics and advertising, and Expo update configuration pointing to u.expo.dev/0c01da2c-5409-459b-a106-98322e3d7076 for over-the-air updates. The RAMADAN_API key is also present for Islamic prayer time calculations. All API communications use the GraphQL endpoint at https://app.willma.life/api/graphql.

9. Feature Summary

Feature	Implementation	Key Details
Barcode Scanning	MLKit TFLite (on-device)	Real-time detection, supports GTIN/UPC/EAN, Code128; product lookup via ScanProduct mutation
Food Photo Scan	Anthropic Claude Opus 4	Image upload + AI analysis; identifies ingredients, estimates portions, calculates macros and health score
Nutrition Label Scan	Same AI pipeline	Extracts structured data from nutrition facts labels; isScannedProduct flag differentiates from AI-estimated data
Premium Gating	GraphQL HasPremiumFeature	Free users limited scans; premium unlimited; paywall after 2 scans (premium-iap-after-two-scans route)
Meal Editing	UpdateScannedMeal mutation	Portion adjustment, ingredient add/remove, macro editing; corrections improve AI over time
Streak System	Local + GraphQL	Tracks daily logging streaks; shields for streak protection; motivational notifications
AI Coach	Anthropic Claude	Context-aware dietary advice using user profile, daily budget, Ramadan timing, and meal history
Product Store	Online store integration	Product recommendations, pricing, brand info based on scanned foods

Table 3: Feature Summary Matrix